Haoxing Quant Assessment — Concepts & Algorithms Reference

A condensed review document of everything you should have top-of-mind going into the 90-minute LeetCode Enterprise assessment. Written assuming you already have the quant/math foundation — this focuses on the algorithmic and implementation side.

Part 1 — Data Structures (know complexities cold)

Arrays & strings

- Access: O(1), append: O(1) amortized, insert/delete at middle: O(n).
- Sorting: `list.sort()` in Python is Timsort, O(n log n).
- Strings in Python are immutable — concatenation in a loop is O(n²). Use `"".join(list_of_strs)` or a list accumulator.

Hash maps (dict) & hash sets (set)

- Lookup, insert, delete: O(1) average, O(n) worst case.
- Iteration: O(n).
- `collections.defaultdict(list)` and `collections.Counter(iterable)` save time.

Linked lists

- Know how to reverse in place, detect cycles (Floyd's tortoise-and-hare), find middle (slow/fast pointers), merge two sorted lists.
- Used in LRU Cache (combined with hash map).

Stacks

- LIFO. Python list with `append()` / `pop()`.
- Used for: balanced parentheses, monotonic stack problems, DFS iterative, expression evaluation.

Queues & deques

- FIFO. Use `collections.deque` — O(1) both ends.
- Used for: BFS, sliding window maximum (monotonic deque).

Heaps (priority queues)

- `heapq` in Python is a **min-heap**. For max-heap, negate values.
- Push/pop: O(log n). Peek (min): O(1). Heapify list: O(n).
- Used for: top-K problems, scheduling, Dijkstra, median of a stream (two heaps).

Trees & binary search trees

- BST ops: O(log n) average, O(n) worst (unbalanced).
- Traversals: inorder (LNR), preorder (NLR), postorder (LRN). Know recursive and iterative forms.
- Balanced trees: AVL, Red-Black — understand concept, rarely need to implement.

Tries

- Prefix tree, O(L) insert/search where L = string length.
- Use when doing many prefix queries. Often overkill; hash maps work for most problems.

Union-Find (Disjoint Set Union)

- Operations: `find` and `union`, both ~O(α(n)) ≈ O(1) with path compression and union by rank.
- Used for: connected components, Kruskal's MST, detecting cycles in undirected graphs.

Graph representations

- **Adjacency list** (`defaultdict(list)` or `list[list[int]]`) — standard choice, O(V+E) space.
- **Adjacency matrix** — $O(V^2)$ space, use only for dense graphs.
- **Edge list** — useful for Bellman-Ford and Kruskal's.

Part 2 — Core Algorithms

Binary search

Two flavors:

Standard binary search (find target in sorted array):

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Binary search on the answer (find minimum/maximum value satisfying a predicate):

```
def min_feasible(lo, hi, feasible):
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid): hi = mid
        else: lo = mid + 1
    return lo
```

Recognize: “minimize the maximum,” “maximize the minimum,” “smallest X such that…”

Two pointers

Two indices moving through one or two arrays. Used for:

- Sorted array targeting a sum (two pointers from both ends).
- Removing duplicates in-place.
- Merging two sorted arrays.
- Partition problems (Dutch national flag).

Sliding window

Maintain a window `[left, right]` that expands on the right and contracts on the left based on a condition.

```
def longest_window(s, condition):
    left = 0
    best = 0
```

```
    state = ...  # e.g., Counter for frequencies
    for right in range(len(s)):
        # add s[right] to state
        while not condition(state):
            # remove s[left] from state
            left += 1
        best = max(best, right - left + 1)
    return best
```

Sorting

- `list.sort(key=..., reverse=...)` for in-place, `sorted(iterable, ...)` for new list.
- Custom comparator: use `functools.cmp_to_key`.
- Know: quicksort average O(n log n), worst O(n²); mergesort always O(n log n); counting sort O(n+k) for bounded integers.

BFS & DFS

BFS with `deque`, for shortest path in unweighted graphs:

```
from collections import deque
def bfs(start, graph):
    visited = {start}
    q = deque([(start, 0)])  # (node, distance)
    while q:
        node, dist = q.popleft()
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                q.append((nb, dist + 1))
```

DFS recursive (watch Python recursion limit, default 1000):

```
def dfs(node, graph, visited):
    if node in visited: return
    visited.add(node)
    for nb in graph[node]:
        dfs(nb, graph, visited)
```

Dijkstra's shortest path

Positive edge weights. O((V+E) log V) with heap.

```
import heapq
def dijkstra(start, graph, n):
    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

Bellman-Ford

Handles negative edge weights, detects negative cycles. O(V·E).

- Key application in quant context: currency arbitrage detection (take –log of exchange rates).

Topological sort

Order a DAG so all edges go forward. Two approaches:

- **Kahn's algorithm** (BFS on in-degrees).
- **DFS with post-order stack**.

Used for course scheduling, build systems, dependency resolution.

Union-Find

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]  # path compression
            x = self.parent[x]
        return x
    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False
        if self.rank[px] < self.rank[py]: px, py = py, px
```

```
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        return True
```

Part 3 — Dynamic Programming Patterns

The meta-pattern: define a state, write the recurrence, decide memoization (top-down) vs tabulation (bottom-up).

1D DP

- **House Robber:** `dp[i] = max(dp[i-1], dp[i-2] + nums[i])`.
- **Climbing Stairs / Fibonacci:** `dp[i] = dp[i-1] + dp[i-2]`.
- **Max subarray (Kadane):** `cur = max(nums[i], cur + nums[i])`, track global max.

2D DP

- **Longest Common Subsequence:** `dp[i][j] = dp[i-1][j-1] + 1` if match, else `max(dp[i-1][j], dp[i][j-1])`.
- **Edit Distance:** three operations → three transitions.
- **Unique Paths:** `dp[i][j] = dp[i-1][j] + dp[i][j-1]`.

State-machine DP (stock/trading problems)

Define state by `(day, holding_stock?, transactions_used)`:

```
# Best Time to Buy and Sell Stock with cooldown
hold = -prices[0]  # holding a share
sold = 0           # just sold (in cooldown next day)
rest = 0           # resting, can buy
for p in prices[1:]:
    prev_sold = sold
    sold = hold + p
    hold = max(hold, rest - p)
    rest = max(rest, prev_sold)
return max(sold, rest)
```

Knapsack patterns

- **0/1 knapsack:** each item once. `dp[j] = max(dp[j], dp[j-w] + v)` with j iterated right-

to-left.

- **Unbounded knapsack:** each item unlimited. Same but j iterated left-to-right.
- **Coin Change (min coins):** unbounded knapsack variant.
- **Partition Equal Subset Sum:** 0/1 knapsack with target = total/2.

Interval DP

- Decide split point. `dp[i][j] = min/max over k of (dp[i][k] + dp[k+1][j] + cost)`.
- Examples: matrix chain multiplication, burst balloons, stone games.

Bitmask DP

- State includes a bitmask of a small set (≤ 20 elements).
- Examples: Traveling Salesman (held-Karp), assignment problems.

DP on trees

- Post-order: compute child answers first, combine at parent.
- Classic: diameter of tree, max sum path, house robber on tree.

Expected value / probability DP

- State: current position or accumulated value.
- Transitions: weighted by probabilities.
- Often solved backward (from absorbing states to initial state).
- Example recurrence: `E[s] = Σ p(s → s') · (reward + E[s'])`.

Part 4 — Probability & Math (your strength — quick refresh)

Key identities

- **Linearity of expectation:** `E[X + Y] = E[X] + E[Y]` always (even if dependent).
- **Variance:** `Var(X) = E[X²] − E[X]²`. `Var(aX + b) = a²·Var(X)`.
- **Covariance:** `Cov(X, Y) = E[XY] − E[X]E[Y]`. `Var(X+Y) = Var(X) + Var(Y) + 2·Cov(X,Y)`.

- **Law of total expectation:** `E[X] = E[E[X | Y]]`.
- **Bayes:** `P(A | B) = P(B | A) · P(A) / P(B)`.

Common distributions

Distribution	Mean	Variance
Bernoulli(p)	p	p(1-p)
Binomial(n, p)	np	np(1-p)
Geometric(p) (trials until 1st success)	1/p	(1-p)/p²
Poisson(λ)	λ	λ
Uniform(a, b)	(a+b)/2	(b-a)²/12
Normal(μ, σ²)	μ	σ²
Exponential(λ)	1/λ	1/λ²

Useful techniques

- **Indicator variables:** E[sum of events] = sum of P(event), by linearity.
- **Reservoir sampling:** sample k items from a stream of unknown length uniformly. Keep first k, for i ≥ k replace a random one with probability k/(i+1).
- **Monte Carlo:** law of large numbers — run many simulations, average results.
- **Fast exponentiation:** `a^n mod p` in O(log n) via repeated squaring.
- **Modular inverse:** when p prime, `a^(-1) ≡ a^(p-2) mod p` (Fermat's little theorem).

Part 5 — Machine Learning Essentials (for this role)

Core operations to implement from scratch

Softmax:

```
def softmax(x):
    # subtract max for numerical stability
    e = np.exp(x - np.max(x, axis=-1, keepdims=True))
```

```
    return e / np.sum(e, axis=-1, keepdims=True)
```

Cross-entropy loss (with one-hot labels y):

```
def cross_entropy(probs, y):
    # clip to avoid log(0)
    return -np.sum(y * np.log(probs + 1e-12)) / probs.shape[0]
```

2-layer MLP forward pass:

```
def mlp_forward(X, W1, b1, W2, b2):
    h = np.maximum(0, X @ W1 + b1)  # ReLU
    logits = h @ W2 + b2
    return softmax(logits)
```

Scaled dot-product attention:

```
def attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    weights = softmax(scores)
    return weights @ V
```

Concepts to know verbally

- **Bias-variance tradeoff:** high bias = underfit, high variance = overfit.
- **Regularization:** L1 (sparsity), L2 (shrinkage), dropout, early stopping.
- **Optimizers:** SGD → Momentum → Adam. Adam adapts learning rate per parameter using running estimates of gradient moments.
- **Activations:** ReLU (standard), sigmoid (binary output), tanh (centered), GELU (transformers), softmax (output layer for multi-class).
- **Loss functions:** MSE for regression, cross-entropy for classification, Huber for robust regression.
- **Train/val/test split:** never tune on test. Use validation for hyperparameter selection.
- **Gradient descent gotchas:** vanishing/exploding gradients, need for normalization (BatchNorm, LayerNorm), learning rate scheduling.

For high-frequency signal research specifically

- **Features matter more than model complexity** in practice — feature engineering is most of the work.
- **Cross-validation must respect time order** — use walk-forward validation, never random k-fold on time series.
- **Sharpe ratio > accuracy** as the target metric in trading contexts.
- **Overfitting is the enemy** — financial data has very low signal-to-noise ratio. Simple models with strong regularization usually beat complex ones.

Part 6 — Time Series / Finance Concepts

Returns

- **Simple return:** `r_t = (P_t - P_{t-1}) / P_{t-1}`.
- **Log return:** `r_t = log(P_t / P_{t-1})`. Preferred because they sum over time: `r_{0→T} = Σ r_t`.

Volatility

- Standard deviation of returns, annualized: `σ_annual = σ_daily · √252` (252 trading days).
- **EWMA volatility:** weights recent observations more; `σ²_t = λ·σ²_{t-1} + (1-λ)·r²_t`.

Performance metrics

- **Sharpe ratio:** `(E[r] - r_f) / σ(r)`. Annualize by multiplying by $\sqrt{252}$.
- **Information ratio:** alpha / tracking error.
- **Max drawdown:** largest peak-to-trough decline.

Time series properties

- **Stationarity:** mean and variance don't change over time. Most classical models require it. Prices are non-stationary; returns usually are.
- **Autocorrelation:** correlation of a series with a lagged version of itself. High autocorrelation in returns would imply predictability (and violate weak-form efficient markets).

- **Volatility clustering:** large moves tend to follow large moves. Motivates GARCH models.

Backtesting pitfalls (likely discussion questions)

- **Look-ahead bias:** using data you wouldn't have had at the time.
- **Survivorship bias:** testing only on companies still listed today.
- **Data snooping:** testing many strategies until one works by chance.
- **Transaction costs:** ignoring them makes any strategy look better.
- **Regime change:** what worked before may not work now.

Part 7 — Algorithmic Complexity Cheat Sheet

Problem size	Acceptable complexity
n ≤ 10	O(n!) — permutation/brute force
n ≤ 20	O(2^n) — bitmask DP, subsets
n ≤ 500	$O(n^3)$ — Floyd-Warshall, interval DP
n ≤ 5,000	$O(n^2)$ — most 2D DP
n ≤ 10^6	O(n log n) — sorting, heap operations
n ≤ 10^8	O(n) — linear scan only
n ≤ 10^{18}	O(log n) — binary search, fast exponentiation

Part 8 — Python-Specific Tips

Built-ins that save time

```
from collections import defaultdict, Counter, deque, OrderedDict
from heapq import heappush, heappop, heapify, nlargest, nsmallest
from itertools import combinations, permutations, accumulate, product
from functools import lru_cache, cmp_to_key
from bisect import bisect_left, bisect_right, insort
```

```
import math
```

Gotchas

- **Integer overflow:** not an issue in Python (arbitrary precision), but watch in other languages.
- **Recursion limit:** default 1000. Use `sys.setrecursionlimit(10**6)` if needed, or convert to iterative.
- **Default mutable arguments:** `def f(x=[])` is a trap — the list is shared across calls.
- **List slicing creates copies:** `arr[i:j]` is O(j-i). Use indices when performance matters.
- **String concatenation in loops:** O(n²). Use `"".join(...)`.

Useful one-liners

- Top-K: `heapq.nlargest(k, iterable)` in O(n log k).
- Unique: `list(set(x))` (unordered) or `list(dict.fromkeys(x))` (preserves order).
- Flatten: `[x for row in grid for x in row]`.
- Transpose: `list(zip(*grid))`.
- Sort by two keys: `sorted(items, key=lambda x: (x[0], -x[1]))`.

Part 9 — The 10-minute pre-test mental checklist

Before clicking "start," spend a few minutes on these:

1. **Environment** — Chrome browser, wired internet if possible, phone silenced, bathroom visit done.
2. **Scratch space** — blank notepad (physical or digital) ready for working out math.
3. **Language** — confirm Python is available; it usually is on leetcode.cn.
4. **Imports** — mentally review the Python imports you'll likely need (collections, heapq, math).
5. **Strategy** — read all problems first, solve easiest to hardest, don't camp on a hard one.
6. **Breathing** — 90 minutes is shorter than it feels. Stay focused but don't rush the read-through.

Final word

You already have the math and stochastic modeling background this firm values — your job over the next few days is to make the coding execution fluent so you can focus mental energy on the problem, not the syntax. Practice in the browser, time-box ruthlessly, and on test day lead with correctness over cleverness.

加油!